

Computergrafik

Skript

Christopher-Manuel Hilgner
christopher.hilgner@stud.hszg.de

Inhaltsverzeichnis

1. Computergrafik Springer	3
2. Bildverarbeitung	3
3. Nutzung und Beispiele	4
4. Begriffe	5
5. Polygone	6
6. 3D Modelle	7
7. Texturen	8
8. Grafikpipeline	9
8.1. OpenGL Pipeline	9
8.1.1. Vertex Processing	9
8.1.2. Vertex Post-Processing	9
8.1.3.	9
9. Globale Rendering Methoden	10
9.1. Raytracing	10
9.2. Radiosity	10
10. Volume Rendering	11
11. Licht & Schatten	12
12. Partikelsysteme	13
13. Optimierungen	14
14. Methoden der Computeranimation	15
15. Voxel	16
16. Hardware	17
16.1. Grafikkarten	17
17. Mathematik für Computergrafik	18
17.1. Grundlagen	18
17.1.1. Lineare Algebra	18
17.1.2. Vektoren	18
17.2. Matrixen	19
17.3. Transforms	20
17.4. Homogene Koordinaten	21
17.5. Coordinate Spaces	22
17.6. Kameras und Projektion	23
17.7. Rasterisierung und Shading	24
17.8. Quaternions & Rotation	25
17.9. Interpolation	26
17.10. Textures, Filtering und Sampling	27
Quellenverzeichnis	29

1. Computergrafik Springer

[1]

2. Bildverarbeitung

[2]

3. Nutzung und Beispiele

4. Begriffe

5. Polygone

6. 3D Modelle

7. Texturen

8. Grafikpipeline

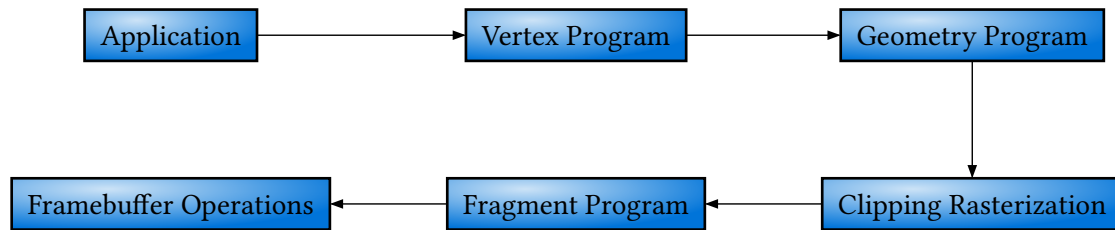


Figure 1: Ablauf einer einfachen Grafikpipeline

8.1. OpenGL Pipeline

[3]

Die Rendering-Pipeline wird ausgeführt, wenn eine Rendering-Operation durchgeführt wird. Damit diese Operation durchgeführt werden kann, muss ein definiertes Vertex Array Objekt und ein Programm Objekt oder Programm Pipeline Objekt für die Shader vorhanden sein.

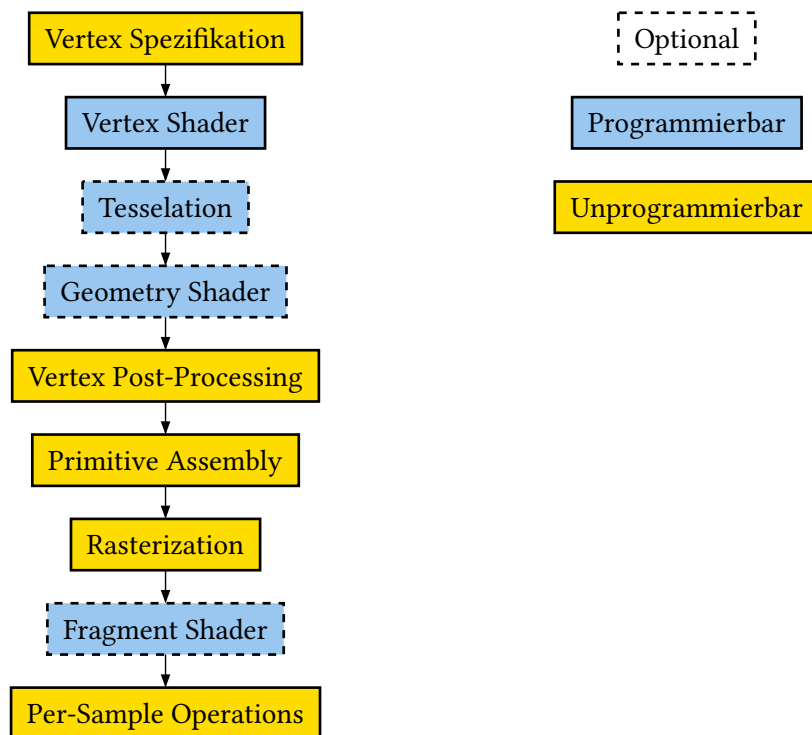


Figure 2:

8.1.1. Vertex Processing

1. Jede Vertex im Vertex Array wird von einem Vertex Shader beeinflusst. Jede Vertex wird dabei zu einer Output Vertex umgewandelt.
2. Optionale primitive Tesselations-Schritte
3. Optionaler Geometrie-Shader. Der Output ist dabei eine Sequenz an Primitiven

8.1.2. Vertex Post-Processing

Die Ausgaben der letzten Stage werden angepasst und zu unterschiedlichen Stellen gebracht.

1. Transform Feedback
2. Primitive Assembly
3. Primitive Clipping, perspective divide, viewport transform zum Window Space

8.1.3.

9. Globale Rendering Methoden

9.1. Raytracing

9.2. Radiosity

10. Volume Rendering

11. Licht & Schatten

12. Partikelsysteme

13. Optimierungen

14. Methoden der Computeranimation

15. Voxel

16. Hardware

16.1. Grafikkarten

17. Mathematik für Computergrafik

17.1. Grundlagen

Themen: Vektoren, Vektor Ops, Dot/Cross Product, Vektor Normalisierung, 3D Geometrie (Punkte, Linien, Ebenen)

Dot Product

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = a + 2b + 3c$$

Cross Product

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} \times \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}$$

Normalisierung

Ein Vektor wird auf eine Größe von 1 skaliert. Dabei wird die Richtung des Vektors beibehalten.

Anwendung: Bewegung eines Spielers mit WASD. Alle Werte des Bewegungsvektors sind dabei auf den Bereich von -1 bis 1 begrenzt. Bewegt sich der Spieler geradeaus, ist der Bewegungsvektor $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$. Somit beträgt die Länge des Vektors auch 1. Bewegt sich der Spieler diagonal mit einem Input von W und D, wäre der Bewegungsvektor $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Die Länge des Vektors wäre somit 1.41 und der Spieler würde sich schneller auf der Diagonalen bewegen. Damit sich der Spieler in alle Richtungen gleich schnell bewegt, muss der Bewegungsvektor normalisiert werden.

Beispiel zur Normalisierung eines Vektors

Normalisierung des Vektors $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ auf die Größe 1. Der Vektor $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ besitzt die Größe 1.41 und soll auf die Größe 1 skaliert werden.

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix} \xrightarrow{\text{normalisierung}} \begin{bmatrix} \frac{1}{1.41} \\ \frac{1}{1.41} \end{bmatrix} = \begin{bmatrix} 0.709 \\ 0.709 \end{bmatrix}$$

Ziele: Positionen repräsentieren, Richtungen, Normalen, Winkeln berechnen, Projektionen, Distanzen

Übungen: Vector Klasse implementieren, Funktionen für dot/cross Produkte schreiben, normalisierung, Einfach Geometrie Probleme lösen (point-plane distance, line-plane distance)

17.1.1. Lineare Algebra

17.1.2. Vektoren

17.2. Matrixen

17.3. Transforms

17.4. Homogene Koordinaten

17.5. Coordinate Spaces

17.6. Kameras und Projektion

17.7. Rasterisierung und Shading

17.8. Quaternions & Rotation

17.9. Interpolation

17.10. Textures, Filtering und Sampling

Quellenverzeichnis

- [1] Alfred Nischwitz, Max Fischer, Peter Haberäcker, and Gudrun Socher, *Computergrafik: Band I des Standardwerks Computergrafik und Bildverarbeitung*. Springer Vieweg, 2018.
- [2] Alfred Nischwitz, Max Fischer, Peter Haberäcker, and Gudrun Socher, *Bildverarbeitung: Band II des Standardwerks Computergrafik und Bildverarbeitung*. Springer Vieweg, 2019.
- [3] “Rendering Pipeline Overview.” [Online]. Available: https://wikis.khronos.org/opengl/Rendering_Pipeline_Overview